

Relatorio de Desempenho

Indice em Arvore B+ na Biblioteca de Persistencia (FrameworkPersistencia)

Comparacao das operacoes CRUD com e sem o indice, para arquivos de 1.000
ate 100.000 registros

Aviso metodologico: os testes foram executados num ambiente Linux isolado (sandbox na nuvem), nao no computador do usuario (Windows/Dev-C++). Os valores absolutos de tempo vao variar numa outra maquina. O que se mantem valido — e e o que este relatorio quer mostrar — e o **padrao relativo**: quais operacoes o indice acelera, quais ele nao acelera, e quais ele deixa mais lentas.

1. Metodologia

Foram comparadas duas versoes da biblioteca de persistencia sobre a mesma struct de teste (cpf, nome, idade), com a mesma sequencia de operacoes e as mesmas chaves aleatorias em ambas, para que a comparacao seja justa:

Com indice — a biblioteca *Persistencia* tal como entregue, usando a arvore B+ (*ArvoreBMais*) internamente em createe, retrieve, update, deletee, queryAll e queryBy.

Sem indice — uma reimplementacao fiel do comportamento original da biblioteca, antes de existir o indice: cada operacao faz varredura linear do arquivo (rewind + fread ate encontrar/terminar), sem nenhuma estrutura auxiliar em memoria.

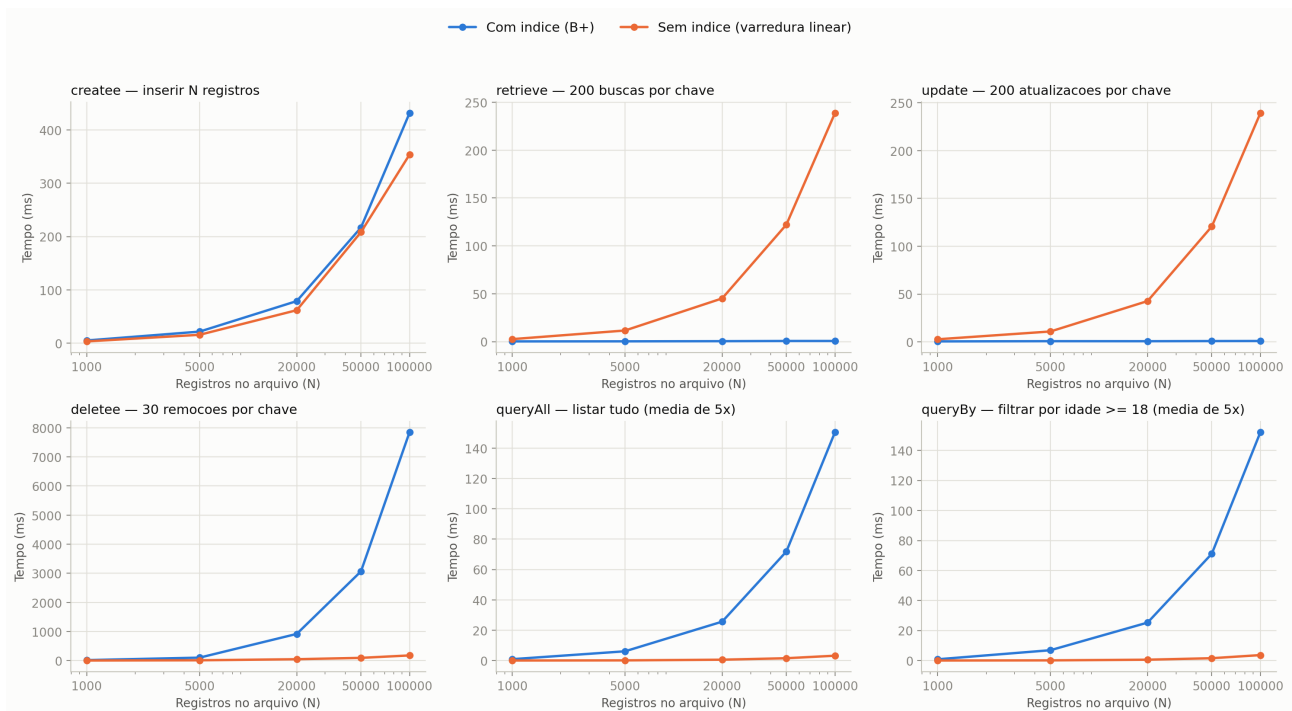
Para cada tamanho de arquivo N (1.000, 5.000, 20.000, 50.000 e 100.000 registros), mediu-se o tempo (em milissegundos, relógio monotônico) de:

Operacao	O que foi medido
createe	Insercao dos N registros (em ordem embaralhada de chave)
retrieve	200 buscas por chave (chaves aleatorias existentes)
update	200 atualizacoes por chave
deletee	30 remocoes por chave (chaves distintas)
queryAll	Listagem de todos os registros (media de 5 execucoes)
queryBy	Filtragem por idade ≥ 18 , predicado sobre o registro inteiro (media de 5 execucoes)

Ambiente: gcc -O2, Linux (sandbox na nuvem), arquivos locais em disco. Cada tamanho N recomeca com arquivos novos (dados e indice apagados antes do teste). Corretude: apos as remocoes, cada rodada confere que o total de registros restantes (via queryAll) e exatamente N menos as 30 remocoes — checagem que passou em todos os tamanhos testados, nas duas versoes.

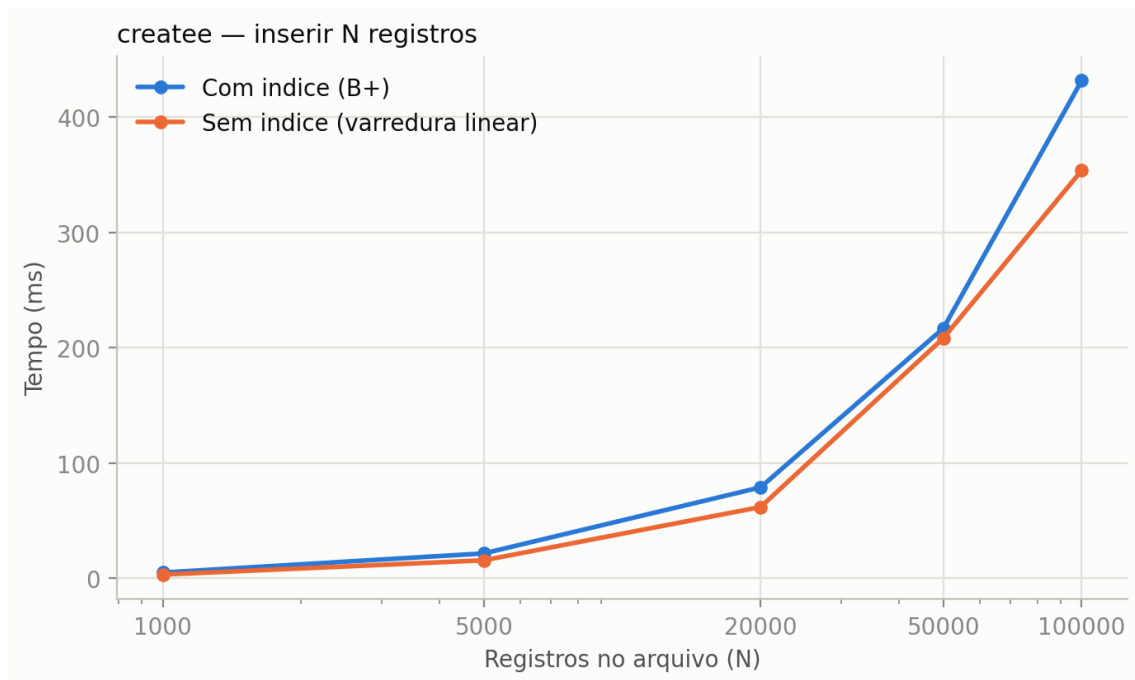
2. Visao Geral

O grafico abaixo resume as seis operacoes lado a lado. Eixo X: numero de registros no arquivo (escala logaritmica). Eixo Y: tempo em milissegundos.



Azul = com indice (arvore B+). Laranja = sem indice (varredura linear).

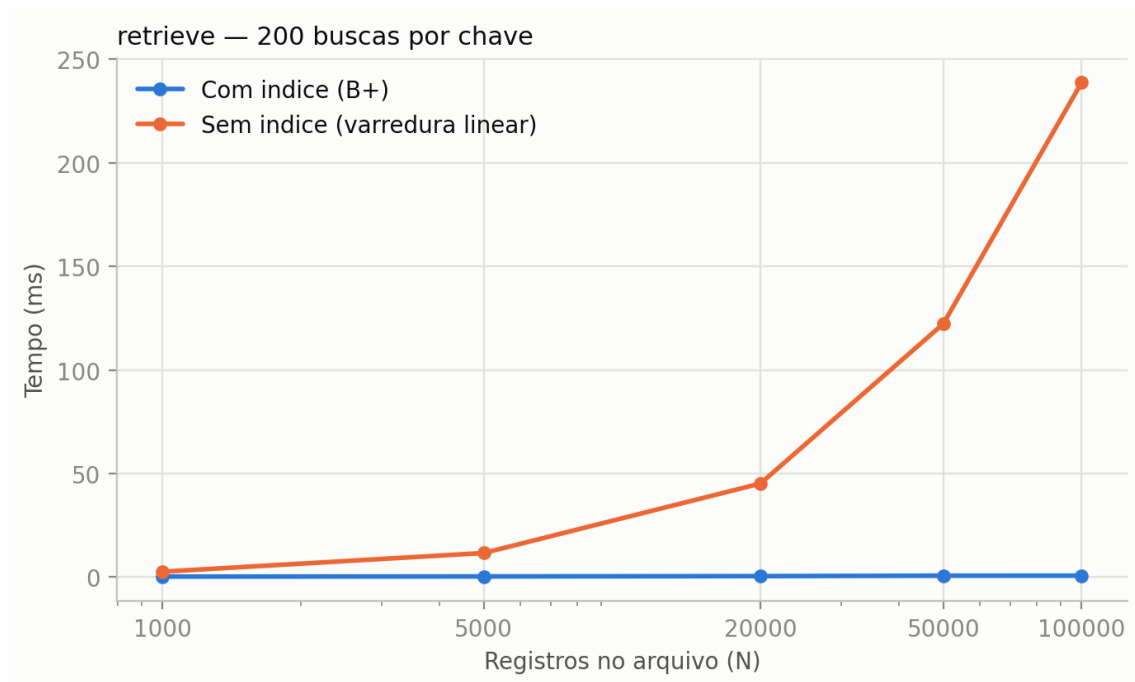
3. createe



N	Com indice (ms)	Sem indice (ms)	Razao (sem / com)
1.000	5.113	3.365	0.66x
5.000	21.733	15.744	0.72x
20.000	78.988	61.924	0.78x
50.000	216.951	208.389	0.96x
100.000	431.734	353.895	0.82x

Cada insercao com indice grava o registro no arquivo (igual a versao sem indice) e **tambem** insere a chave na arvore B+ ($O(\log N)$). Por isso o indice tem um custo extra aqui, nao um ganho: e o preco que se paga para as buscas ficarem rapidas depois.

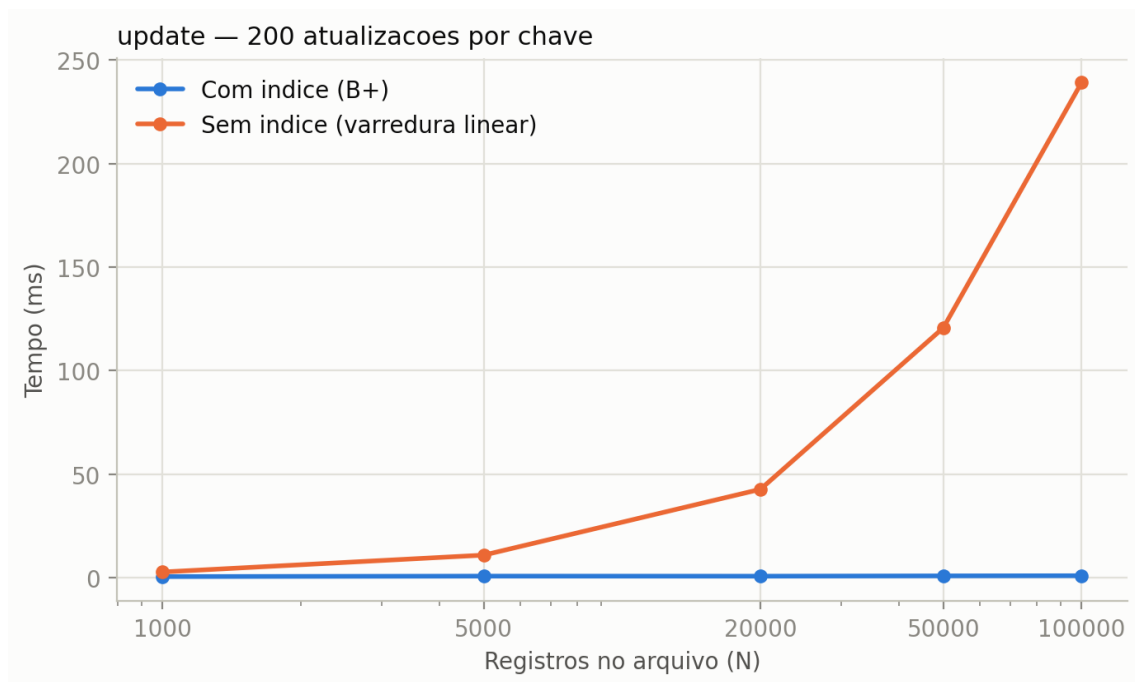
4. retrieve



N	Com indice (ms)	Sem indice (ms)	Razao (sem / com)
1.000	0.226	2.590	11.48x
5.000	0.295	11.607	39.28x
20.000	0.457	45.139	98.82x
50.000	0.658	122.242	185.86x
100.000	0.668	238.717	357.58x

Este é o caso clássico de uso de índice: sem ele, cada busca precisa ler o arquivo inteiro até achar (ou não achar) a chave — custo que cresce com N . Com a árvore B+, a busca desce $O(\log N)$ níveis e vai direto ao offset do registro no arquivo.

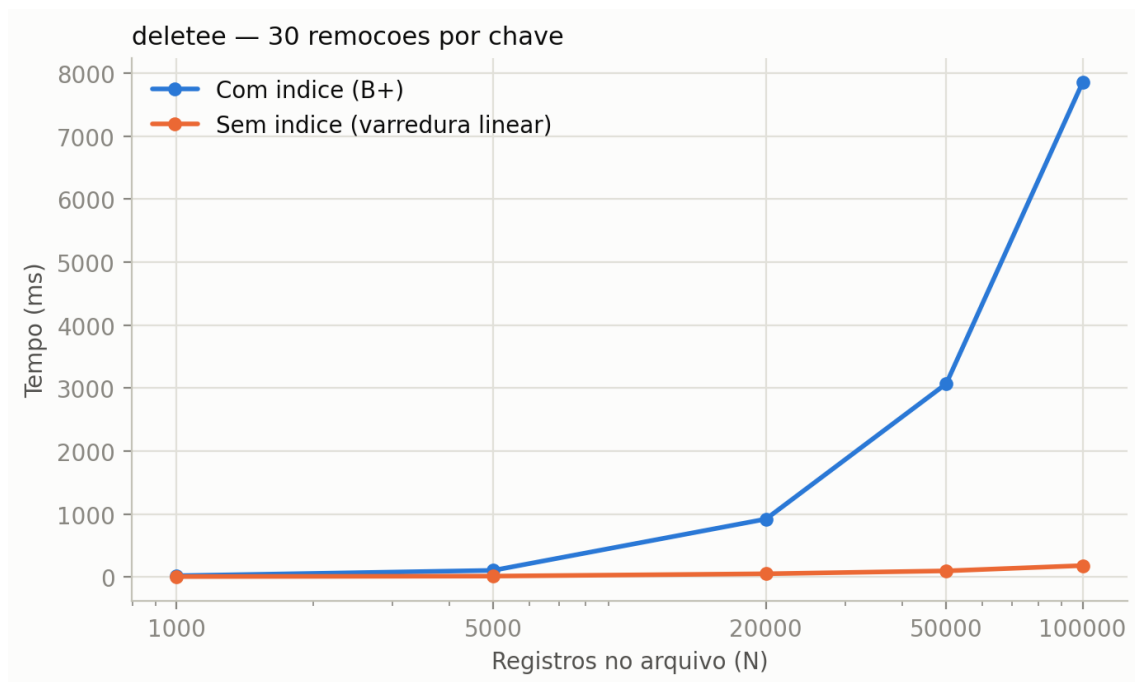
5. update



N	Com indice (ms)	Sem indice (ms)	Razao (sem / com)
1.000	0.523	2.735	5.23x
5.000	0.755	10.895	14.43x
20.000	0.723	42.692	59.09x
50.000	0.867	120.690	139.17x
100.000	0.943	239.202	253.69x

update usa retrieve internamente para localizar o registro antes de sobrescrever, entao herda o mesmo ganho: praticamente constante com indice, crescente sem ele.

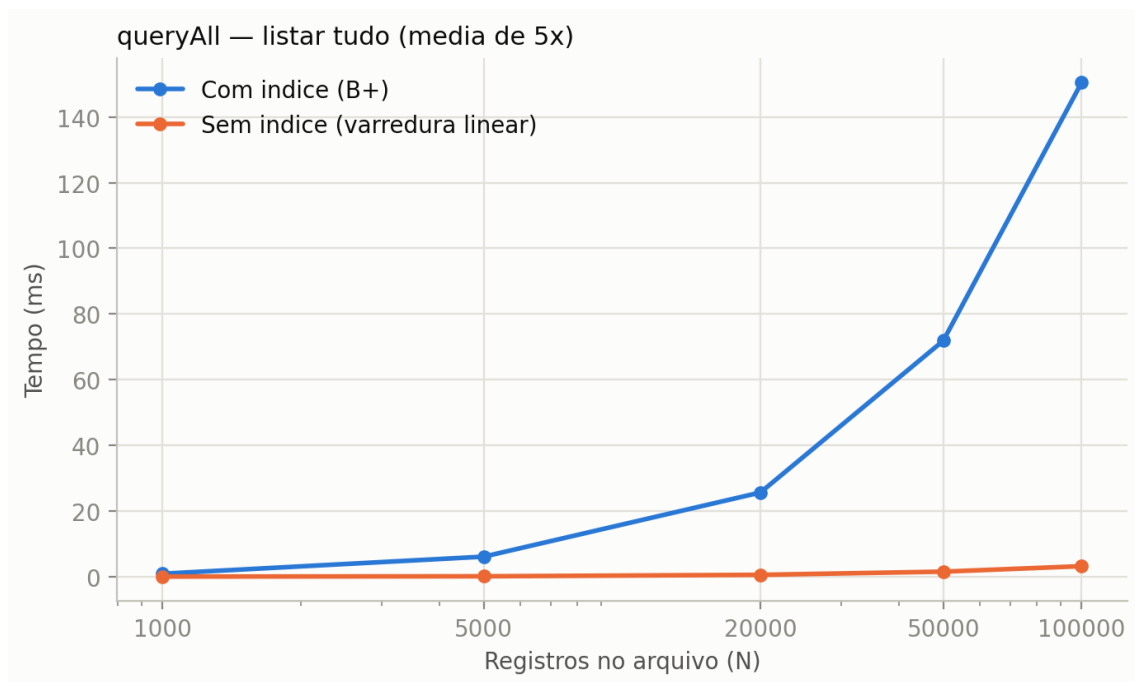
6. deletee



N	Com indice (ms)	Sem indice (ms)	Razao (sem / com)
1.000	19.576	5.967	0.30x
5.000	104.601	13.731	0.13x
20.000	919.424	50.742	0.06x
50.000	3067.010	96.720	0.03x
100.000	7847.870	180.549	0.02x

Aqui o indice sai **pior**, e vale destacar o motivo: como o delete reescreve o arquivo inteiro (os offsets de todos os sobreviventes mudam), a implementacao atual reconstrói a arvore B+ inteira do zero a cada remocao, alem de reescrever o arquivo. Isso soma um custo $O(N \log N)$ de reindexacao ao custo $O(N)$ de reescrita que ja existia sem indice — e explica a curva azul crescer muito mais rapido. Fica registrado como uma oportunidade de otimizacao futura (por exemplo, atualizar so as entradas afetadas, em vez de reconstruir a arvore inteira).

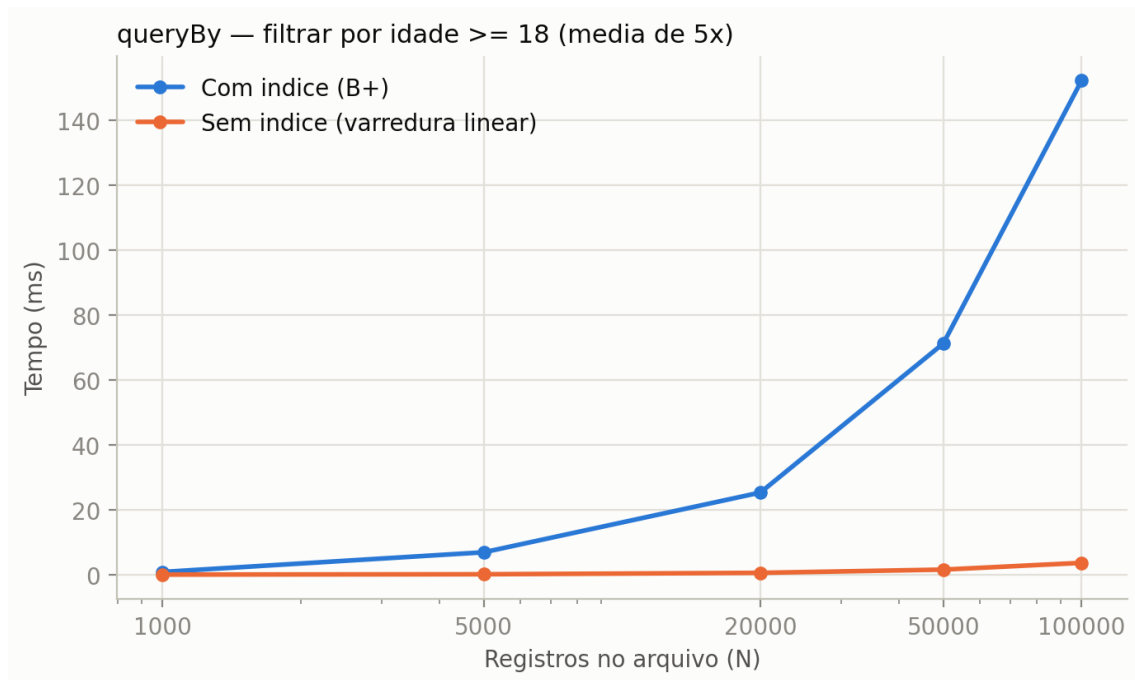
7. queryAll



N	Com indice (ms)	Sem indice (ms)	Razao (sem / com)
1.000	0.901	0.023	0.03x
5.000	6.094	0.115	0.02x
20.000	25.636	0.560	0.02x
50.000	71.865	1.529	0.02x
100.000	150.501	3.191	0.02x

Também sai pior com índice, por um motivo diferente: percorrer a árvore de acesso aos registros em ordem de chave, o que significa um fseek + fread por registro (potencialmente fora de ordem no arquivo). A varredura sequencial sem índice lê o arquivo de forma contínua, o que o sistema operacional consegue otimizar muito bem. Como queryAll precisa mesmo ler todos os registros, o índice não reduz o trabalho de fundo — só muda o padrão de acesso ao disco, e nesse caso para pior.

8. queryBy



N	Com índice (ms)	Sem índice (ms)	Razao (sem / com)
1.000	0.843	0.025	0.03x
5.000	6.891	0.124	0.02x
20.000	25.297	0.564	0.02x
50.000	71.181	1.558	0.02x
100.000	152.177	3.638	0.02x

Mesma explicacao de queryAll: o predicado (idade ≥ 18) e avaliado sobre o registro inteiro, nao sobre a chave indexada, entao todo registro precisa ser lido de qualquer forma. O indice nao pode pular nenhum registro aqui — ele so piora o padrao de leitura.

9. Conclusao

O indice em arvore B+ cumpre exatamente o papel esperado de um indice de banco de dados: acelera drasticamente buscas por chave (retrieve/update), ao custo de um pequeno overhead na insercao (createe). Mas os dados deixam claro que **indice nao e sinonimo de "mais rapido em tudo"**:

- **queryAll e queryBy** nao se beneficiam de um indice de chave unica, porque precisam ler todos os registros de qualquer forma (e, no caso de queryBy, filtram por um campo que nao e a chave indexada). Nesta implementacao, elas ficaram mais lentas com indice, porque trocaram uma leitura sequencial do arquivo por leituras via offset (potencialmente fora de ordem).

- **deletee** ficou visivelmente mais lento com indice, por reconstruir a arvore inteira a cada remocao — um custo de implementacao, nao uma limitacao inerente de indices B+.

Em resumo: o indice vale a pena quando o padrao de uso e dominado por buscas/atualizacoes por chave. Se o uso real for dominado por listagens completas ou filtros sobre outros campos, o custo extra de manter o indice pode nao compensar — a menos que as rotinas de queryAll/queryBy/deletee sejam otimizadas para tirar melhor proveito dele (por exemplo, atualizando o indice incrementalmente no delete, em vez de reconstrui-lo).